HUSKEY POCKET

Documentación Técnica del Firmware

Arquitectura, módulos, interfaz y funciones

Referencia de desarrollo — plataforma Waveshare ESP32-S3-Touch-LCD-2.8

Contenido

Contenido 2
1. Panorama general 3
2. Organización del código 3
3. Ciclo de vida de la aplicación 4
4. Módulo vault.h — núcleo criptográfico y de datos 5
5. Módulos vault_csv.h y vault_hky.h — importación y exportación 7
6. Canales de salida hacia el host 8
7. Periféricos 9
8. Interfaz de usuario 10
9. Notas de seguridad transversales 11

1. Panorama general

Este documento describe la estructura interna del firmware de Huskey Pocket: cómo se organiza el código, qué hace cada módulo, y cuál es la interfaz pública de las funciones principales. Está pensado como referencia de desarrollo, complementaria al whitepaper de seguridad (que cubre el porqué de cada decisión). Aquí el foco es el cómo.

El firmware es un proyecto Arduino/ESP-IDF para el ESP32-S3, escrito en C++ y organizado en archivos de cabecera (.h) que agrupan funcionalidad por dominio, más el sketch principal (.ino) que orquesta el arranque y el bucle. La interfaz gráfica se construye con LVGL. El almacenamiento cifrado se apoya en mbedTLS, NVS y LittleFS.

Plataforma y dependencias

Elemento	Detalle
MCU / placa	ESP32-S3 sobre Waveshare ESP32-S3-Touch-LCD-2.8
Framework	Arduino-ESP32 (núcleo 3.x)
Interfaz gráfica	LVGL
Criptografía	mbedTLS (AES, PBKDF2, HMAC-SHA)
Almacenamiento	NVS (Preferences) + LittleFS
Bluetooth	NimBLE-Arduino 2.x (perfil HID de teclado)
USB	TinyUSB (HID nativo y almacenamiento masivo)
Sensores	RTC PCF85063ATL, IMU QMI8658C (bus I2C compartido)

Constantes globales de referencia

Constante	Valor y significado
`PIN_DIGITS`	6 — longitud del PIN
`VAULT_PBKDF2_ITERATIONS`	50 000 — iteraciones PBKDF2 del PIN del dispositivo
`HKY_ITERATIONS`	100 000 — iteraciones PBKDF2 de la exportación cifrada
`VAULT_SALT_LEN`	16 — bytes de salt
`VAULT_FIELD_LEN`	64 — tamaño máximo de nombre, usuario, contraseña y secreto
`VAULT_NOTES_LEN`	128 — tamaño máximo de las notas
`VAULT_MAX_ENTRIES`	3500 — entradas máximas del vault
`VAULT_MAX_CATS`	16 — categorías máximas

2. Organización del código

El código se agrupa en cuatro capas conceptuales. Un mismo archivo puede pertenecer a una sola capa; el .ino las conecta.

Capa de núcleo y datos

- password_manager.ino — punto de entrada: setup() y loop(). Decide el modo de arranque (normal o almacenamiento USB), inicializa hardware y periféricos, y despacha los ticks de fondo (Snatch Lock, autobloqueo).

- vault.h — el corazón del sistema: cifrado, derivación de clave, verificación de PIN, PIN señuelo, FRP, Modo perdido, categorías, TOTP, generación de contraseñas y toda la API de lectura/escritura del vault.
- vault_csv.h — exportación e importación en CSV (formato propio y de navegadores), con analizador RFC 4180 e importación incremental.
- vault_hky.h — exportación e importación cifrada (contenedor .hky) en streaming, apoyada en el mismo analizador CSV.

Capa de periféricos

- rtc_manager.h — reloj de tiempo real dedicado (PCF85063ATL).
- snatch_lock.h — Bloqueo por arrebato mediante el IMU (QMI8658C).
- sd_manager.h — montaje de la tarjeta SD, lectura de líneas, borrado seguro y utilidades de archivo.
- battery_manager.h, power_manager.h, vibro_manager.h — batería, gestión de energía/Deep Sleep y feedback háptico.
- Display_ST7789.*, Touch_CST328.* — controladores de pantalla y táctil.

Capa de entrada/salida hacia el host

- ble_keyboard_manager.h — teclado HID por Bluetooth (NimBLE) y emparejamiento.
- usb_keyboard_manager.h — teclado HID por USB nativo.
- usb_msc_manager.h — modo de almacenamiento masivo USB (expone la SD al PC).
- kb_manager.h — capa de abstracción que enruta el tecleado al canal activo (BLE o USB).

Capa de interfaz de usuario

- pin_screen.h, lock_screen.h, onboarding.h — pantallas de bloqueo, PIN y configuración inicial.
- ui_main.h, ui_list.h, ui_forms.h, ui_settings.h — pantalla principal, lista de entradas, formularios y ajustes.
- ui_generator.h, ui_qr.h, ui_statusbar.h, power_ui.h — generador de contraseñas, códigos QR, barra de estado e interfaz de energía.
- custom_kb.h, custom_kb_t9.h — teclados en pantalla (completo y T9).
- i18n.h, colors.h, fonts.h — internacionalización (ES/EN), paleta de temas y fuentes.

3. Ciclo de vida de la aplicación

Arranque (setup)

El arranque decide primero si el dispositivo debe iniciar en modo de almacenamiento USB. Esa decisión se toma antes de cargar el vault o cualquier otra funcionalidad, leyendo una bandera persistente:

```
if (vault_get_usb_msc_mode()) {     // Inicializa LVGL mínimo y arranca usb_msc_begin();
return;   // modo exclusivo: no se carga el vault }
```

En arranque normal, la secuencia es: inicializar pantalla y táctil, montar la SD y limpiar exports residuales, ejecutar la verificación de Modo perdido (que puede disparar un restablecimiento de fábrica antes de pedir el PIN), y mostrar la pantalla de bloqueo/PIN. El Bluetooth no se inicia aquí: solo arranca tras un desbloqueo correcto.

Bucle principal (loop)

El loop atiende LVGL (interfaz y táctil) y ejecuta los ticks de fondo. El más relevante para seguridad es el del Bloqueo por arrebato, que lee el IMU en cada vuelta si está activo y el vault desbloqueado.

Desbloqueo

Al validar el PIN, on_unlock() desbloquea la sesión, arranca el Bluetooth si el modo de teclado es BLE, y carga la pantalla principal. La clave derivada vive en RAM a partir de este punto.

Bloqueo

El bloqueo (manual, por autobloqueo, por arrebato o por re-lock) limpia la clave de la RAM, cierra el Bluetooth y devuelve a la pantalla de PIN o entra en Deep Sleep, según el disparador.

4. Módulo vault.h — núcleo criptográfico y de datos

Concentra el modelo de seguridad y el acceso a datos. Sus responsabilidades: derivar claves, verificar PINs, cifrar y descifrar campos, gestionar el vault señuelo, FRP, Modo perdido, categorías, TOTP y la generación de contraseñas.

Cifrado de campos

Cada campo sensible se cifra individualmente con AES-256-CBC. El cifrado antepone un IV aleatorio de 16 bytes al ciphertext y aplica relleno PKCS7. El descifrado valida que la longitud sea múltiplo del bloque y que el relleno sea íntegro antes de aceptar los datos.

Función interna	Propósito
`_derive_key_into(pin, salt_path, out)`	Deriva una clave de 32 bytes por PBKDF2 hacia un buffer explícito, sin tocar la clave de sesión.
`_aes_encrypt / _aes_decrypt`	Cifran/descifran un campo con la clave de sesión, gestionando IV y relleno.
`_write_control / _verify_control`	Escriben y verifican el «valor de control» cifrado que confirma si un PIN es correcto sin almacenar el PIN.

Autenticación — API pública

Función	Descripción
`bool vault_init(pin)`	Deriva ambas claves candidatas (señuelo y real) en tiempo constante, abre el vault correspondiente y marca la sesión activa. Registra el timestamp de desbloqueo real para el Modo perdido.
`void vault_lock()`	Limpia la clave de la RAM y cierra la sesión.
`bool vault_verify_pin(pin)`	Confirma un PIN (real o señuelo) sin alterar la sesión activa; se usa antes de acciones sensibles.
`bool vault_is_ready()`	Indica si hay un vault desbloqueado.
`bool vault_is_decoy()`	Indica si la sesión activa es el vault señuelo.

Factory Reset Protection (FRP)

Función	Descripción
`vault_frp_save_pin(pin)`	Guarda el PIN de FRP con su propio salt, en un namespace que no se borra en el factory reset.
`vault_frp_verify(pin)`	Verifica el PIN de FRP.
`vault_frp_get/set/reset_attempts()`	Gestionan el contador persistente de intentos de FRP.
`vault_frp_lock_permanent()`	Marca el bloqueo permanente tras agotar los intentos.
`vault_frp_is_locked_permanent()`	Consulta el estado de bloqueo permanente.

Contador de intentos y restablecimiento

Función	Descripción
`vault_get/set/reset_fail_count()`	Gestionan el contador persistente de intentos fallidos del PIN, reservado antes de cada verificación.
`vault_factory_reset()`	Borra vault, categorías, preferencias, contador y residuos del señuelo; preserva el registro de FRP; pone la clave en cero.

Vault señuelo, Modo perdido y Snatch Lock

Función	Descripción
`vault_decoy_set_pin / clear_pin / is_set()`	Configuran, eliminan y consultan el PIN señuelo.
`vault_get/set_deadman_days()`	Configuran los días del Modo perdido (0 = desactivado).
`vault_deadman_check()`	Verifica en cada arranque si procede el borrado por Modo perdido, con protección ante RTC hacia atrás.
`vault_get/set_snatch_level()`	Configuran el nivel de sensibilidad del Bloqueo por arrebato (0-3).

Entradas del vault

Función	Descripción
`vault_save_web(name, user, pass, cat...)`	Crea o edita una entrada de tipo web; trunca de forma segura al tamaño de campo.
`vault_save_secret(name, secret, cat)`	Crea o edita una entrada de tipo secreto.
`vault_load(name)`	Carga y descifra una entrada por nombre.
`vault_list(names, count, max, cat)`	Lista los nombres de entradas, opcionalmente filtrados por categoría.
`vault_count(cat, type)`	Cuenta entradas, con filtros opcionales.
`vault_delete(name)`	Elimina una entrada.
`vault_set_favorite(name, fav)`	Marca o desmarca una entrada como favorita.

TOTP y generación de contraseñas

Función	Descripción
`vault_totp_code(name, seconds_left)`	Calcula el código TOTP de 6 dígitos de una entrada. No registra el secreto ni el código por diagnóstico.
`vault_generate_password(out, len, upper, nums, syms, no_ambig)`	Genera una contraseña con el RNG por hardware, muestreo sin sesgo y longitud acotada al buffer.

Función	Descripción
`vault_get/set_gen_config()`	Persisten la configuración por defecto del generador.

> *El almacenamiento del vault usa una tabla hash sobre LittleFS con sondeo lineal. Cada operación de localización abre y descifra registros a lo largo de la cadena de sondeo, por lo que el rendimiento se degrada a medida que la tabla se acerca a su capacidad. Es un compromiso conocido del backend.*

5. Módulos vault_csv.h y vault_hky.h — importación y exportación

vault_csv.h

Implementa la exportación y la importación en CSV. El analizador respeta el formato RFC 4180 (campos entre comillas, comillas dobles escapadas) y detecta automáticamente el separador y el formato del encabezado (propio de Huskey o de navegador). La importación es incremental: procesa unas pocas líneas por llamada desde un temporizador de la interfaz, de modo que la pantalla táctil sigue respondiendo durante importaciones grandes.

Función	Descripción
`vault_export_csv(path)`	Exporta el vault a un CSV en la SD.
`vault_import_csv_begin(path)`	Cuenta líneas, detecta formato y prepara la importación incremental.
`vault_import_csv_step(batch)`	Procesa hasta «batch» líneas; se detiene de forma ordenada si el vault se llena.
`vault_import_csv_progress(...)`	Reporta progreso: importadas, errores, líneas procesadas y total.
`vault_import_csv_cancel()`	Cancela una importación en curso.

vault_hky.h

Implementa el contenedor cifrado .hky. La exportación deriva una clave de la contraseña de exportación (independiente del PIN) y escribe una cabecera con formato, salt, iteraciones e IV, seguida del contenido cifrado en streaming, comenzando por un bloque de verificación fijo. La importación descifra sobre la marcha y alimenta el mismo analizador CSV, sin materializar nunca el CSV completo en claro. Rechaza archivos cuya cabecera declare menos iteraciones que el mínimo.

Función	Descripción
`vault_export_hky(path, password)`	Exporta el vault cifrado con la contraseña dada.
`vault_import_hky_begin(path, password)`	Abre y valida el contenedor, prepara el descifrado en streaming.
`vault_import_hky_cleanup(path)`	Limpia de forma segura los archivos temporales tras la importación.

6. Canales de salida hacia el host

ble_keyboard_manager.h — teclado Bluetooth

Emula un teclado HID por BLE con NimBLE. El emparejamiento usa confirmación numérica: se muestra un código de 6 dígitos que el usuario confirma en pantalla. Las pulsaciones solo se transmiten sobre un enlace cifrado y autenticado.

Función	Descripción
`ble_kb_begin() / ble_kb_end()`	Inician y detienen la pila BLE y el advertising.
`ble_kb_connected()`	Indica si hay una conexión física activa.
`ble_kb_secure()`	Indica si el enlace está además cifrado y autenticado; requisito para teclear.
`ble_type_string / ble_type_user / ble_type_pass / ble_type_secret`	Tecleaan cadenas o campos concretos hacia el host.
`ble_type_tab() / ble_type_enter()`	Envían las teclas Tab y Enter.
`ble_kb_num_bonds()`	Consulta el número de vínculos guardados.
`ble_kb_delete_all_bonds()`	Elimina todos los vínculos de forma efectiva (pila y almacenamiento).

usb_keyboard_manager.h — teclado USB

Emula un teclado HID por USB nativo mediante TinyUSB. Expone una interfaz análoga a la del BLE (usb_type_string, usb_type_user, usb_type_pass, usb_type_secret, usb_type_tab, usb_type_enter). El descriptor HID se entrega al framework con el tamaño exacto declarado.

kb_manager.h — enrutado

Capa fina que enruta las funciones de tecleado al canal activo (BLE o USB) según la preferencia del usuario, de modo que el resto de la interfaz no necesita saber qué transporte está en uso.

usb_msc_manager.h — almacenamiento masivo

Expone la tarjeta SD al host como una unidad USB. Es un modo exclusivo: no coexiste con el vault ni con los teclados. Los accesos de bloque validan el rango contra la capacidad de la tarjeta y rechazan transferencias mal formadas; existe un modo de solo-lectura opcional. La partición del vault, al estar en la Flash interna, nunca queda expuesta.

Función	Descripción
`usb_msc_begin()`	Arranca el modo de almacenamiento masivo y muestra la pantalla mínima.
`vault_get/set_usb_msc_mode()`	Consultan y fijan la bandera persistente que activa el modo al arrancar.
`vault_get/set_usb_msc_readonly()`	Consultan y fijan la preferencia de solo-lectura.

7. Periféricos

rtc_manager.h — reloj de tiempo real

Controla el RTC dedicado PCF85063ATL por I2C. Mantiene la hora aunque el dispositivo pierda alimentación principal, gracias a su batería propia. Es la única fuente de tiempo del vault, usada por TOTP y Modo perdido. Las escrituras validan todos los rangos y las lecturas enmascaran los bits de estado.

Función	Descripción
`rtc_begin()`	Inicializa el chip, limpia el bit de parada y detecta pérdida de hora.
`rtc_set(year, month, day, hour, min, sec)`	Ajusta la hora con validación de rangos.
`rtc_is_set()`	Indica si el RTC tiene una hora válida.
`rtc_get_time_str / rtc_get_date_str`	Devuelven cadenas formateadas de hora y fecha.
`rtc_get_epoch()`	Devuelve un timestamp para los cálculos del vault.

snatch_lock.h — Bloqueo por arrebato

Lee el IMU QMI8658C por software en cada vuelta del bucle. Calcula la magnitud combinada de aceleración y giro; si ambas superan el umbral del nivel elegido de forma simultánea, entra en Deep Sleep de inmediato. Exigir ambas condiciones a la vez reduce los falsos positivos. No actúa mientras hay una operación sensible en curso.

Función	Descripción
`snatch_lock_init()`	Configura el IMU (rangos, frecuencia de muestreo) y verifica su presencia.
`snatch_lock_tick()`	Se llama en cada vuelta del bucle; evalúa el arrebato si el nivel está activo y el vault desbloqueado.

sd_manager.h — tarjeta SD

Monta la SD y ofrece utilidades de archivo: lectura de líneas (con descarte controlado de líneas sobredimensionadas), borrado seguro de archivos y creación de las carpetas de import/export. El borrado seguro sobrescribe antes de eliminar, con las limitaciones físicas conocidas de los medios Flash.

power_manager.h, battery_manager.h, vibro_manager.h

- power_manager.h — gestión de energía y entrada/salida de Deep Sleep, apagando panel, táctil y SD antes de dormir.
- battery_manager.h — lectura del nivel de batería.
- vibro_manager.h — feedback háptico (vibración) en las interacciones.

8. Interfaz de usuario

La interfaz se construye con LVGL. Se organiza por pantallas, cada una en su propio archivo, más los componentes compartidos (teclados en pantalla, barra de estado, temas y textos).

Pantallas de acceso

Módulo	Función
onboarding.h	Configuración inicial: fija el PIN, la fecha/hora y las preferencias base la primera vez.
pin_screen.h	Pantalla de PIN: entrada de dígitos, manejo de intentos, flujo de FRP y del temporizador del señuelo.
lock_screen.h	Pantalla de bloqueo y transición hacia la de PIN.

Pantallas principales

Módulo	Función
ui_main.h	Pantalla principal tras el desbloqueo; navegación y accesos rápidos.
ui_list.h	Lista de entradas: búsqueda, filtrado por categoría, selección múltiple, detalle y acciones sobre cada entrada.
ui_forms.h	Formularios de creación y edición de entradas.
ui_settings.h	Ajustes: seguridad, teclado, idioma, tema, modos USB/BLE, import/export, Modo perdido y Bloqueo por arrebato.

Componentes y utilidades

Módulo	Función
ui_generator.h	Generador de contraseñas con opciones de longitud y conjuntos de caracteres.
ui_qr.h	Presentación de datos como código QR.
ui_statusbar.h	Barra de estado (hora, batería, indicadores).
power_ui.h	Interfaz relacionada con energía y apagado.
custom_kb.h / custom_kb_t9.h	Teclados en pantalla: completo y T9.
i18n.h	Internacionalización ES/EN mediante tablas indexadas por clave.
colors.h / fonts.h	Paleta de temas y definiciones de fuentes.

9. Notas de seguridad transversales

Algunas invariantes de seguridad no viven en un solo módulo sino que se sostienen en varios puntos del código. Conviene tenerlas presentes al modificar cualquier parte:

- La clave derivada del PIN solo existe en RAM y se limpia explícitamente en cada bloqueo y restablecimiento. Cualquier ruta nueva que abra o cierre sesión debe respetar esa limpieza.
- El vault señuelo nunca debe persistir datos reales: las funciones de guardado comprueban el modo señuelo y simulan éxito sin escribir. Toda función de escritura nueva debe hacer lo mismo.
- Ningún material sensible (PIN, contraseñas, secretos, claves, códigos TOTP) debe registrarse por el puerto serie ni por ningún otro canal de diagnóstico.
- Las verificaciones de PIN real y señuelo deben mantener tiempo constante entre sí para no filtrar cuál se probó.
- Los contadores de intentos (PIN y FRP) deben reservarse de forma persistente antes de verificar, no después.
- Toda entrada externa (archivos importados, datos recibidos por USB/BLE) es no confiable y debe validarse en tamaño y rango antes de usarse.

> *Este documento describe el estado del firmware tras la auditoría de seguridad. Las decisiones de diseño y sus límites se detallan en el whitepaper. Flash Encryption y Secure Boot, que refuerzan el escenario de acceso físico, siguen pendientes de forma deliberada hasta congelar la funcionalidad.*